CS 6362 Advanced Machine Learning, Fall 2026
Assignment 2: Understanding Deep Learning Requires Rethinking Generalization

Souvik Ghosh

October 21, 2026

1 Problem Statement and Methodology

1.1 Problem Statement

We have a training set $S = \{(x_i, y_i)\}_{i=1}^n$ drawn i.i.d. from an unknown distribution $\mathcal{D}$, where $x_i \in \mathbb{R}^d$ is an image and $y_i \in \{1, \ldots, K\}$ is its class ($K = 10$ for CIFAR10). A network f_θ with parameters $\theta \in \mathbb{R}^p$ is trained with SGD to minimize the empirical risk

$$\hat{L}_S(\theta) = \frac{1}{n} \sum_{i=1}^{n} \ell(f_\theta(x_i), y_i), \tag{1}$$

and we care about the population risk $L_\mathcal{D}(\theta) = \mathbb{E}_{(x,y)\sim\mathcal{D}}[\ell(f_\theta(x), y)]$. The generalization gap is $L_\mathcal{D}(\theta) - \hat{L}_S(\theta)$. With the 0–1 loss and zero training error, this is just the test error.

The networks used in practice have $p \gg n$, but they still have a small gap on real data. The question the paper asks is: what explains this? Is it the model class (VC dimension, Rademacher complexity), the regularizers, or something about the training algorithm?

1.2 Model and Methodology

The main tool is a randomization test. They take a standard architecture (Inception, AlexNet, MLPs on CIFAR10, and Inception V3 on ImageNet) and train it on modified copies of the data without changing the model or the hyperparameters:

- **Partially corrupted labels:** each label y_i is replaced, with probability p, by a class drawn uniformly from $\{1, \ldots, K\}$. The corrupted labels $\tilde{y}_i$ are drawn once and then kept fixed. $p = 1$ gives completely random labels.
- **Shuffled pixels:** one fixed permutation of the pixels is applied to every image (train and test).
- **Random pixels:** each image gets its own random permutation.
- **Gaussian:** images are replaced by Gaussian noise that has the same mean and variance as the dataset.

In all these cases the link between x and y is partly or completely destroyed, so nothing can be learned that holds on the test set. Even so, the networks reach zero training error. Training on random labels is only slower by a small constant factor. As p goes from 0 to 1, the test error rises smoothly towards chance level.

They also turn the explicit regularizers (data augmentation, weight decay, dropout) on and off. The models still generalize well without them, so regularization helps but does not explain generalization on its own.

Finally, there are two smaller theoretical parts. The first is a construction showing that a two-layer ReLU network with $2n + d$ weights can fit any labeling of n points in d dimensions. The second is a linear-model argument showing that SGD started at zero lands on the minimum-norm solution, which is a form of implicit regularization.

1.3 Mathematical Derivations and Intuition

The cleanest way to see why random labels are a problem for classical theory is through Rademacher complexity. For a hypothesis class $\mathcal{H}$ and data $\{x_1, \dots, x_n\}$,

$$\hat{\mathfrak{R}}_n(\mathcal{H}) = \mathbb{E}_\sigma \left[\sup_{h \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^{n} \sigma_i h(x_i) \right], \qquad \sigma_i \in \{\pm 1\} \text{ i.i.d. uniform.} \tag{2}$$

This is literally "how well can the class fit random ± 1 labels". A typical bound looks like

$$L_{\mathcal{D}}(h) \le \hat{L}_S(h) + 2\,\hat{\mathfrak{R}}_n(\mathcal{H}) + O\!\left(\sqrt{\tfrac{\log(1/\delta)}{n}}\right). \tag{3}$$

If the network can fit any random labeling, then $\hat{\mathfrak{R}}_n(\mathcal{H}) \approx 1$ and the bound says nothing. The key point is that the model, the data size and the optimizer are exactly the same in the true-label and the random-label runs. So any bound that depends only on $\mathcal{H}$ and n has to be the same for both, even though one has about 10% test error and the other has 90%. The same argument works for VC dimension. It also works for uniform stability, since that is a property of the algorithm that does not look at the labels.

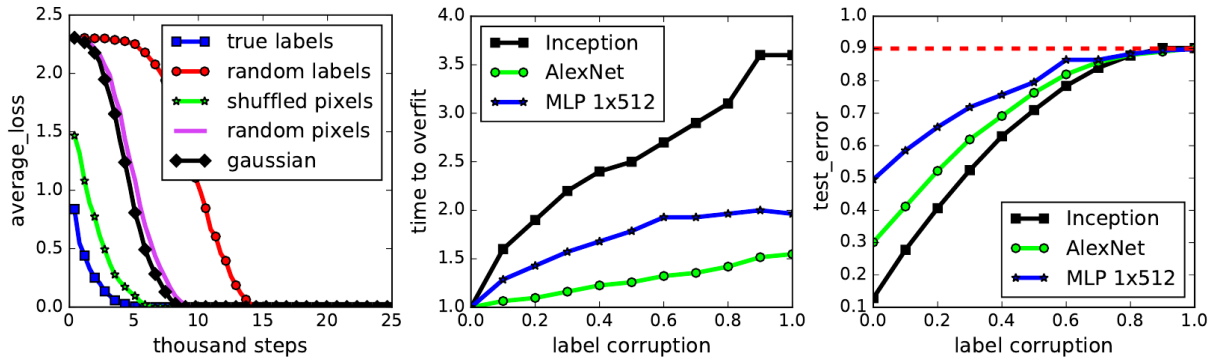
For the linear case, if $w_0 = 0$ then every SGD update is a combination of data points, so $w = X^\top \alpha$. If we also fit the training data, $Xw = y$, then

$$X X^\top \alpha = y, \tag{4}$$

which has a unique solution and gives the minimum ℓ_2-norm interpolant. The intuition is that SGD picks a "simple" solution among the many that fit the data. The authors also point out that norm alone does not predict test error, so this is only part of the story.

2 Reproduction of Figure 1 (a, b, c)

2.1 Original Figure



(a) learning curves (b) convergence slowdown (c) generalization error growth

Figure 1: Figure 1 of Zhang et al. (2017), taken from the paper. (a) Training loss of the small Inception under five data settings. (b) Time to overfit, relative to the uncorrupted run, against the label corruption level p. (c) Test error against p.

2.2 LLM Details and Code

- **LLM (name and version):** Claude Opus 5.5 (claude.ai chat interface)
- **Shared conversation:** https://claude.ai/share/01dd9cfc-6e8f-45b3-a71e-2f518fb224f3
- **GitHub repository:** https://github.com/souvik7701/ML-Assignment-2. The notebook `Paper_1.ipynb` reproduces the figure, and the saved results of all runs are included.

During Phases 1–3 the LLM was not given any existing code and did not look for any. I ran all the training myself on a Colab T4 GPU.

2.3 Phase 1: Prompting the LLM with the Paper

I gave the LLM the paper PDF and asked it to reproduce Figure 1 in PyTorch, split into pieces I could run one by one in Colab. I told it not to use, pull or even look at any existing implementation, and to work only from the paper.

The code it returned follows Appendix A for the optimizer, learning-rate schedule and preprocessing. It builds the five data settings and the three models, runs the experiments for all three panels, and plots the result. It marked every detail the paper does not give as an assumption, such as the batch size, padding, weight initialization and a definition of "time to overfit" (full list under Implementation Choices).

One thing it did well was to use the parameter counts in Table 1 to pin down details the text leaves out. A BatchNorm with a learnable shift but no scale gives the paper's count for Inception, and 64 filters per convolution gives the count for AlexNet. All three models in my run have the same number of parameters as in the paper.

2.4 Phase 2: Reviewing the LLM Output

The code ran without errors. Panel (c) looked right, but two problems showed up in the figure.

Jagged loss curves in (a). The curves had a sawtooth with a period of one epoch. This is not a training bug. The data is reshuffled every epoch, and on random labels the network lowers the loss only by memorizing individual samples. Samples late in an epoch were, on average, last seen longer ago and have been "forgotten" more, so the loss rises within each epoch and drops at each reshuffle. The logging windows do not line up with the epochs, so each window catches a different part of this cycle. As expected, the effect is strongest for random labels and Gaussian inputs, and weakest for true labels.

Badly defined "time to overfit" in (b). The LLM measured it as the first epoch at which full training accuracy reaches 99%. This has two problems. First, accuracy was only checked at epoch boundaries, and Inception fits true labels in only a few epochs, so this rounding alone shifts the $p = 0$ baseline noticeably. Since every ratio is divided by that baseline, this rescales the whole curve. Second, 99% sits right at the ceiling that some models reach on random labels in the paper's own Table 1. Near that ceiling the measured time is dominated by a slow final stretch. In my first plot the AlexNet curve shot up at high corruption.

2.5 Phase 3: Good-Faith Reproduction

I kept the data, models and training code as they were. The changes were all in the plotting code, so no runs had to be repeated:

- The training loss is averaged over a sliding window of one epoch, and each point is placed at the center of its window so the curve is not shifted in time. This removes the sawtooth.
- "Time to overfit" is now the first step at which this smoothed loss falls below a threshold, which is much finer than one epoch.
- The LLM suggested a threshold of 0.05. I raised it to 0.1, because the MLP's run on true labels stops just above 0.05, so with 0.05 the MLP has no baseline and its whole curve disappears.
- Since the paper does not define the threshold, the plot also reports how the $p = 1$ ratio depends on it (Section 2.8).

I then asked the LLM why the model ordering in (b) is reversed. Its answer was that the learning-rate schedule runs out for AlexNet and the MLP. I tested this explanation afterwards (Section 2.8).

2.6 Phase 4: Comparison with an Existing Implementation

The original code belongs to Google and was never released. The first author, Chiyuan Zhang, has a public demo repository (`https://github.com/pluskid/fitting-random-labels`). Its README says it is written from scratch, has no association with the paper's code, and does not try to reproduce the paper's experiments. It uses a Wide ResNet, not Inception or AlexNet. So only the label corruption and the MLP can be compared directly.

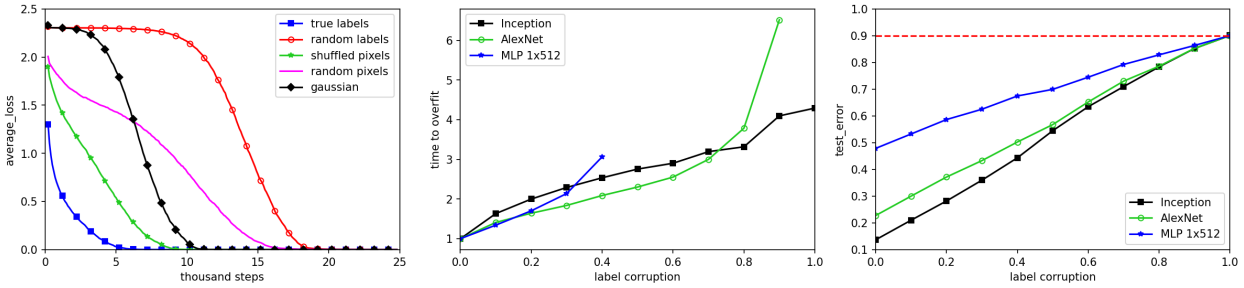
The label corruption works the same way as mine: labels are corrupted once, with a fixed seed, and kept for all epochs. The batch size matches my guess, and there is no data augmentation.

The most important difference is the learning-rate schedule. The repo keeps the rate constant for most of training and then drops it in steps, while the paper and my code decay it every epoch. In

the README's own log, the MLP needs far more epochs to fit random labels than the paper's decay allows (see below). The repo's MLP also starts from larger weights (Glorot initialization instead of PyTorch's default). The other differences matter less for this figure: different preprocessing, weight decay by default, and corrupted test labels, whereas Figure 1c needs the true test labels.

I made no code changes after this comparison. The two main differences, a much longer learning-rate budget and a larger initialization, point at the same cause as my diagnosis below. I did not adopt them, because the paper states the 0.95 per-epoch decay.

2.7 Reproduced Figure



(a) learning curves (b) convergence slowdown (c) generalization error growth

Figure 2: My reproduction of Figure 1 (single seed). In (b), points are missing where the smoothed training loss never reached 0.1 before the run stopped (AlexNet at $p = 1$, MLP at $p \geq 0.5$).

Panel (c) matches well. At $p = 0$ the test errors are 0.137 (Inception), 0.227 (AlexNet) and 0.479 (MLP 1x512). The paper's Table 1 gives 0.143, 0.239 and 0.495. All three curves rise to about 0.90 at $p = 1$, with the same ordering as the paper (Inception lowest, MLP highest). As in the paper, AlexNet and Inception come together at high corruption.

Panel (a) matches qualitatively. The ordering is the same: true labels < shuffled pixels < Gaussian < random pixels < random labels. Random labels show the plateau near $\ln 10 \approx 2.30$, then a fast drop to zero. Everything is somewhat slower than in the paper, though (random labels reach zero at about 18k steps, against about 14k). One clear difference is the random-pixel curve. In the paper it starts on the plateau at 2.3 and drops close to the Gaussian curve. Mine starts at 2.0 and falls slowly. My guess is that this comes from the LLM's assumption that a "pixel" is a spatial location whose RGB values move together. Under that assumption every permuted image keeps its own color histogram, which still carries some class information that the network can use from the first step. The paper may instead have permuted all color values independently.

Panel (b) matches only for Inception. The Inception curve rises steadily to 4.3 at $p = 1$, against 3.6 in the paper. AlexNet follows a similar curve up to $p = 0.8$, then jumps to 6.5 at $p = 0.9$ and has no point at $p = 1$. The MLP has no points above $p = 0.4$. In the paper the order at $p = 1$ is Inception (3.6) > MLP (2.0) > AlexNet (1.5), so for AlexNet and the MLP my result goes the other way.

2.8 Failures, Diagnosis, and Corrections

The learning-rate budget. With $\eta_t = \eta_0 \gamma^t$ per epoch and $\gamma = 0.95$, the total step size available over all of training is

$$\sum_{t=0}^{\infty} \eta_0 \gamma^t = \frac{\eta_0}{1-\gamma} = 20\,\eta_0, \tag{5}$$

and after E epochs a fraction $1 - \gamma^E$ of it is already used (about two thirds after 20 epochs, and almost all of it by epoch 80). So however long a run trains, it can only move about as far as 20 epochs at the initial rate. If a run needs a fraction f of this budget to reach the threshold, it gets there at epoch

$$E(f) = \frac{\ln(1-f)}{\ln \gamma}, \tag{6}$$

which blows up as $f \to 1$. The intuition is that a model with plenty of spare budget shows how hard memorization is, while for a model near the limit the measured time is mostly the slow final stretch where the learning rate has almost vanished. Then small differences in difficulty turn into huge ratios, or into runs that never reach the threshold. Inception starts with a rate ten times higher (0.1) and has BatchNorm. AlexNet and the MLP start at 0.01 without BatchNorm, and PyTorch's default initialization is small for ReLU networks, which slows them down further.

Evidence. I tested this explanation with a short diagnostic at the end of the notebook. It needs no retraining, since it only reads the logs of the saved runs. Table 1 shows selected rows.

model	p	epochs	fit epoch	budget at fit	final loss	train acc
Inception	0.0	16	10	42%	0.006	99.99%
	1.0	48	45	90%	0.017	99.97%
AlexNet	0.0	18	14	52%	0.015	99.97%
	0.9	120	93	99%	0.067	99.75%
	1.0	120	–	–	0.761	79.42%
MLP 1x512	0.0	39	28	76%	0.051	99.90%
	1.0	120	–	–	0.154	99.16%

Table 1: Learning-rate budget diagnostic (selected runs). Runs stop at 99.9% training accuracy or at 120 epochs. "Fit epoch" is when the smoothed loss first reaches 0.1, and "budget at fit" is the fraction of the total learning rate used by then.

The table supports the explanation. Inception always stops on accuracy, within 50 epochs, although at $p = 1$ it has used 90% of the budget and is not far from the limit either. AlexNet behaves normally up to $p = 0.8$. At $p = 0.9$ it only reaches the threshold at epoch 93, when almost the whole budget is gone, which is the jump to 6.5 in panel (b). At $p = 1$ it fails to fit: after 120 epochs it is at 79% training accuracy, against 99.8% in the paper, and its loss is still falling with less than 2% of the budget left. The MLP runs to the epoch cap for every $p \geq 0.2$, and at high corruption its loss levels off just above the threshold. Its accuracy at $p = 1$ is still close to the paper's, so by accuracy it behaves like the paper's MLP. It is missing from panel (b) only because its loss never gets down to 0.1.

The Inception ratio also depends on the threshold. At $p = 1$ it goes from 5.3 to 3.4 as the threshold goes from 0.2 down to 0.02. The paper's 3.6 lies in this range, so for Inception the definition of "time to overfit" alone can explain the mismatch.

Why my result differs from the paper. The paper says that "we do not need to change the learning rate schedule". For the two models without BatchNorm, my runs show that the schedule is the main limit. The paper half admits this in Appendix E: to fit random labels with data augmentation, the authors had to slow down the decay (which they call the "weight decay factor", probably a typo). Since their AlexNet nearly fit random labels with the same schedule, their runs must have made more progress per unit of learning rate than mine. The likely reasons are details the paper does not give: weight initialization (TensorFlow defaults differ from PyTorch's), batch size (a smaller batch means more steps per epoch before the rate decays) and the LRN constants. The paper's definition of "time to overfit" is also unknown. The MLP never reaches 100% in their Table 1 either, so their measure cannot have been "100% training accuracy", and it may have been relative, for example when the loss levels off. Other smaller differences are mixed precision training and using a single seed with no error bars.

Checks I propose but did not run. Re-running AlexNet and the MLP at $p = 0$ and $p = 1$ with He initialization, or with a slower decay, would separate initialization from the schedule. If the explanation is right, both changes should let AlexNet fit random labels and bring its ratio down into the paper's range. The Phase 4 comparison (Glorot init and a 150-epoch constant rate in the author's own demo) points in the same direction.

2.9 Required Questions

How are random labels generated across epochs, and what would go wrong otherwise? The corrupted labels are drawn once, before training, with a fixed seed, and then stay the same for every epoch. The paper stresses this ("the random labels are fixed and consistent across epochs"). If new labels were drawn every epoch, there would be nothing consistent to memorize. Each image would get a fresh label on every visit, so the best the network could do is predict the uniform distribution. The loss would stay near $\ln 10 \approx 2.30$ and training accuracy near 10%. The experiment would then test nothing about memorization capacity, since no model could fit the labels.

With corruption probability p, what fraction of labels actually differ from the truth, and what test error should you expect at $p = 1$? Each label is resampled with probability p, uniformly over all $K = 10$ classes, and the resampled label equals the true one with probability $1/K$. So

$$\Pr[\tilde{y}_i \neq y_i] = p\left(1 - \frac{1}{K}\right) = 0.9\,p. \tag{7}$$

At $p = 1$ about 90% of the labels are wrong, not 100%. The training labels then carry no information about the images, so whatever the network memorizes is independent of the true test label. Its prediction on a test image matches the true class with probability $1/K$, and the expected test error is $1 - 1/K = 90\%$, which is what all three of my models get.

What optimizer, learning-rate schedule, preprocessing and regularization do the authors use?

- **Optimizer:** SGD with momentum 0.9.
- **Learning rate:** 0.1 for small Inception and 0.01 for small AlexNet and the MLPs, multiplied by 0.95 after every epoch. The batch size is not given.

- **Preprocessing:** divide pixels by 255, center crop to 28×28, then TensorFlow's per-image whitening, which subtracts each image's mean and divides by $\max(\sigma, 1/\sqrt{2352})$.
- **Regularization:** none for the randomized-label and pixel experiments, so no weight decay, dropout or data augmentation.

2.10 Implementation Choices

- **Convergence time (Fig. 1b).** The first step at which the one-epoch moving average of the minibatch training loss is ≤ 0.1, divided by the same quantity for the $p = 0$ run of the same model. Runs stop at 99.9% training accuracy (evaluated on the full training set after each epoch) or after 120 epochs. A missing point means the threshold was never reached. The sensitivity to the threshold is reported in Section 2.8.
- **Grid.** I did not reduce it. Panels (b) and (c) use all 11 corruption levels $p = 0, 0.1, \ldots, 1$ for all three models (Inception, AlexNet, MLP 1x512), which is 33 runs. Panel (a) uses Inception on the five settings for 25k steps each.
- **Other assumptions.** Batch size 128 (391 steps per epoch); PyTorch default initialization; "same" padding; BatchNorm with shift but no scale in Inception; 64 filters and the original AlexNet LRN constants in AlexNet; original labels for the input-corruption settings; a "pixel" means a spatial location, so its RGB values move together; a Gaussian with per-channel dataset mean and standard deviation; mixed precision (fp16) for speed; one training seed (0) per run.

3 Critical Review

3.1 Strengths

The experiment is very simple, and that is why it is convincing. Changing only the labels and keeping everything else fixed is a clean control, and it is hard to argue with the result. The authors run it on several architectures and on ImageNet, so it is not a quirk of one model. The paper is also easy to read. It separates the different possible explanations (model class, explicit regularization, implicit regularization) and tests each one. The finite-sample expressivity result is short and fits well with the experiments.

3.2 Weaknesses

The paper is mostly a negative result. It does a good job of showing what does *not* explain generalization, but it does not give a real alternative. Section 5 on implicit regularization is the closest it comes, but it only covers linear models, and the authors then show that the minimum-norm idea does not predict performance either (the wavelet example on MNIST). So the title promises more than the paper delivers.

The argument also only rules out bounds that depend on the hypothesis class alone. Bounds that depend on the data and the trained solution, such as margin-based, norm-based or PAC-Bayes bounds, are not ruled out. These are mentioned briefly, but not tested with the same care as the randomization experiments.

On the experimental side, most results are from a single run with no error bars, and there is no hyperparameter tuning for the random-label case. That is fine for the main point, but it makes the exact shape of the curves in Figure 1 hard to trust.

3.3 Potential Errors

- The "time to overfit" in Figure 1b is not clearly defined. It is plotted relative to the true-label run, but the paper does not say which threshold counts as having fit the data.
- The claim that regularization is "not necessary" is a bit stronger than the data supports. AlexNet with weight decay fails to fit random labels, so in that case regularization does change what can be fit.
- In Table 2, the best test accuracy along training is reported in parentheses as a measure of early stopping. This picks the stopping point on the test set itself, which the authors admit in a footnote.

3.4 Comparison with OpenReview Reviews

The paper got very high scores: 10, 9 and 10 from the three reviewers, and it was accepted as an oral. The program chairs called the experiments "a model of thoroughness". The reviewers liked the same things I did: the clever use of randomization and the clear argument against VC dimension, Rademacher complexity and stability.

The main criticism also matches mine. Reviewers 1 and 3 both said the paper gives little explanation of *why* over-parameterized networks generalize, and no clear direction for what to do next. Reviewer 1 asked why networks with the same number of parameters generalize differently, and the authors replied that this is "the most important open question in machine learning". Reviewer 2 asked them to state clearly that the random labels come from the same label set. In the rebuttal the authors clarified that the labels are drawn once and kept fixed across epochs, which is the detail that matters most for reproduction.

The official reviewers did not raise my points about data-dependent bounds, single runs or the vague "time to overfit". Some public comments were more critical. One showed that 1-nearest-neighbour and random forests also fit random labels easily. Another (Thomas Dietterich) pointed out that in Figure 1a random labels take more than twice as many steps as true labels, which is hardly a "small" slowdown. That is close to my concern about Figure 1b. A third comment argued that the min-norm result in Section 5 comes from the zero initialization rather than from SGD itself. Overall I was a bit more critical than the official reviewers, but we agree on the main strength and the main weakness.

References

[1] C. Zhang, S. Bengio, M. Hardt, B. Recht, and O. Vinyals. Understanding deep learning requires rethinking generalization. In *International Conference on Learning Representations*, 2017.